

Programmation et Algorithmique I

Corrigé du test du 31 octobre 2012

Remarque

Ce document contient *une* solution pour chaque problème du test de novembre. Bien sûr, les algorithmes pour résoudre un problème ne sont pas uniques et votre algorithme, s'il est différent, est peut-être correct également.

Problème 1 : quel est le mot de passe ?

```
def crack_password(pwlength):
    pwd = ''
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    length = 0
    pwdFound = False
    while not pwdFound and length <= pwlength:
        i = 0
        while not(is_letter_in_position(length, alphabet[i])):
            i = i + 1
        pwd = pwd + alphabet[i]
        if perfect_match(pwd):
            pwdFound = True
        length = length+1
    return pwd
```

Erreurs fréquentes :

- Instruction totalement inutile : `if True`.
- Mauvaise initialisation de `pwd` : `pwd = "*" * pwlength` (au lieu d'une chaîne vide construite petit à petit).
- Ne pas utiliser `perfect_match` fera continuer la boucle jusqu'à `pwlength` ce qui peut poser problème si le mot de passe est plus court (par ex. `pwlength=8` et le mot de passe est `maison` (6<8)).
- Evaluer `is_letter_in_position` avec toutes les lettres de l'alphabet même si la lettre a déjà été trouvée (par exemple si le `a` a été trouvé on continue à tester avec `b,c,d, ...`).

Problème 2 : jeu de dominos

```
def inverse(d):
    return (d[1], d[0])

def conciliable(d1, d2):
    return d1[1] == d2[0] or d1[1] == d2[1]

def compatible(d1, d2):
    return d1[1] == d2[0]

def retirer(dominos, d):
    di = inverse(d)
    new_dominos = []
    removed = False
    for el in dominos:
        if not removed and (el == d or el == di):
            removed = True
        else:
            new_dominos.append(el)
    return new_dominos

def suite_dominos(dominos, debut):
    if len(dominos) == 0:
        return [debut]
    else:
        for el in dominos:
            if conciliable(debut, el):
                if compatible(debut, el):
                    candidate = el
                else:
                    candidate = inverse(el)
                suite = suite_dominos(retirer(dominos, candidate), candidate)
                if suite != None:
                    return [debut] + suite
        return None
```

Erreurs fréquentes :

- a) `inverser(domino)`
 - Ecrire `domino = x, y` au lieu de `x, y = domino` (affectation de gauche à droite).
 - (pas incorrect mais inutilement long) Inverser avec une variable temporaire :

```
x, y = domino
temp = x
x = y
y = temp
return x, y
```

alors que `return y, x` après la première ligne ferait l'affaire.
 - Créer une liste `[x, y]` au lieu d'un couple `(x, y)`
 - Modifier un tuple (qui est immuable) : `temp = d[0]` puis `d[0] = d[1]` puis `d[1] = temp`.
- b) `est_compatible(d1, d2)` et `est_conciliable(d1, d2)`
 - (pas incorrect mais inutilement long) Au lieu d'écrire `return x` (où `x` est un booléen), écrire :

- ```

 if x == True:
 return True
 else:
 return False

```
- Attention à lire correctement l'énoncé pour faire les bonnes comparaisons (beaucoup d'erreurs à cause de cela).
- c) `jouer(dominos, d)`
- Pas de création de nouvelle liste, ou alors modification de l'ancienne avec effet de bord (genre : `new_liste = dominos`).
  - Souvent, l'erreur classique est de ne pas se limiter à retirer la première occurrence mais de retirer toutes les occurrences. Ou alors, quand la boucle fait un "append" dans la liste résultat, s'arrêter dès que l'occurrence à supprimer est trouvée et oublier de recopier les éléments après.
  - Sur une liste `L` : `L.pop(d)` où `d` est l'élément à supprimer au lieu d'un indice.
- d) `suite_dominos(dominos, debut)`
- Considérer que `debut` est une liste des dominos qui composent le "début de la suite". L'exemple dans l'énoncé (et l'énoncé en soit) stipule pourtant bien le contraire.
  - Faire l'appel récursif "dans le vent", sur une ligne seule sans stocker le résultat. Certains étudiants ajoutent un `if True` juste après (voir remarque similaire pour le problème 1).
  - Une erreur très courante est de ne pas tester le cas `suite_dominos(...) == None` et de considérer que `suite_dominos` retourne toujours une liste (sans doute parce que le cas de base, `dominos = []`, n'a qu'une seule issue possible et que le "`return None`" se fait dans le cas récursif).
  - Le cas de base `len(dominos) == 0` est plus simple que d'utiliser `len(dominos) == 1` (où il ne faut pas oublier de vérifier que le seul domino restant est conciliable avec début, ou d'autres cas de base).
  - S'arrêter dès que `suite_dominos` retourne `None`, au lieu de simplement passer au domino suivant dans ce cas.
  - Pour éviter un effet de bord, certains étudiants font parfois une copie (correcte ou non) de la liste `dominos`. Soit via `deepcopy`, soit via un slice, ou via une copie de référence (ce qui est une erreur). Cette copie est inutile, puisque la fonction "jouer" permet justement de récupérer cette nouvelle liste. C'est d'ailleurs une erreur courante que de ne pas utiliser les fonctions implémentées juste avant.

### Problème 3 : approximation d'une abscisse

```

def find_abs(f,d,a,b,epsilon):
 c_old = a
 c = (b+a)/2.
 while abs(c_old - c)>epsilon:
 if f(c) < d:
 a = c
 else:
 b = c
 c_old = c
 c = (a+b)/2.
 return c

```

#### Erreurs fréquentes :

- Certains étudiants ont remis un algorithme récursif, alors qu'un itératif est explicitement demandé.

- A chaque fois que l'on va évaluer une nouvelle approximation (qui sera stockée dans la variable `c`), nous stockons dans `c_old` la dernière approximation (contenue dans la variable `c` avant qu'elle ne soit modifiée). Cela permet de conserver les deux dernières approximations calculées pour évaluer la condition d'arrêt. En effet, il était demandé de sortir de la boucle lorsque la distance entre deux approximations successives était inférieure à `epsilon` (paramètre de la fonction). Certains étudiants n'ont pas utilisé cette variable `c_old` et ont appliqué la condition d'arrêt suivante : "`b-a < epsilon`". Cette dernière ne respecte pas l'énoncé dans le sens où si `a`, par exemple, est modifié lors de deux itérations consécutives, alors "`b-a`" ne représente pas la différence de deux approximations consécutives.
- Toujours concernant la condition d'arrêt, une erreur fréquente est de comparer `f(c)` avec `d` qui sont des ordonnées et non des approximations de l'abscisse recherchée.
- Concernant le calcul d'une nouvelle approximation (instruction `(a+b)/2.`), beaucoup d'étudiants ont d'une part oublié le problème de la division entière et d'autre part oublié de modifier la valeur de `a` ou `b` en fonction du nouvel intervalle choisi. Ce qui rend erronée l'instruction "`(a+b)/2.`" permettant de calculer l'approximation suivante.

## Problème 4 : une élection étrange

- a) *Preuve d'arrêt.* Pour vérifier que la fonction `depouillement` se termine toujours, il faut vérifier que la seule boucle qu'elle contient se termine après un nombre fini d'itérations. Il s'agit d'une boucle `while` dont la condition est `len(urne) > 1`. Ainsi, si la liste possède un seul élément, elle n'est pas (ou plus) exécutée. Supposons qu'elle contienne  $n > 1$  éléments avant une itération donnée. On observe que (dans le cas où `vote1 == vote2` mais également dans l'autre cas), il y a, lors de l'exécution de l'itération, trois modifications de la liste : deux appels à `pop` et un appel à `append`. Cela signifie que, après l'exécution de l'itération complète, il y a 2 éléments retirés et 1 élément ajouté, et donc au final  $n - 1$  éléments dans la liste. Il s'ensuit qu'après un nombre fini d'itérations il restera exactement 1 élément dans la liste et la boucle se terminera.
- b) *Preuve de l'invariant.*
- *Avant d'entrer dans la boucle.* Pour rappel,  $x$  représente le nombre de B au départ et  $y$  le nombre de B courant. Ainsi, avant d'entrer dans la boucle,  $x$  est égal à  $y$  et l'assertion

$$S \equiv x \text{ et } y \text{ ont la même parité}$$

est trivialement vérifiée.

- *Conservation de l'invariant après une itération complète.* On suppose que l'invariant est vrai après avoir exécuté  $k \geq 0$  itérations. Est-il préservé après avoir exécuté la  $k + 1$ ème itération ? Un analyse du code montre qu'il y a plusieurs cas à analyser en fonction de la valeur des 2 votes tirés.
  - Deux votes A sont tirés : ils sont retirés de l'urne et un A est remplacé. Le nombre de B ne change donc pas. En conséquence, la parité de  $y$  est inchangée et reste donc identique à celle  $x$  par hypothèse d'induction. L'invariant est donc toujours respecté.
  - Deux votes B sont tirés : ils sont retirés de l'urne et un A est ajouté dans l'urne. Le nombre de B diminue donc de 2, ce qui préserve également la parité de  $y$ . Par les mêmes arguments que dans le cas précédent, l'invariant est toujours respecté.
  - Un vote A et un vote B sont tirés : ils sont retirés de l'urne et un B est remplacé, ce qui implique que le nombre de B et la parité de  $y$  restent inchangés.
- c) *Preuve de la validité de la règle (\*).* La règle stipule que si le nombre initial de bulletins pour B est pair, alors A est élu. Dans l'autre cas (le nombre de B est impair), c'est B qui est élu. Observons d'abord que la fin de la preuve d'arrêt énoncée ci-dessus (point a)) implique qu'après l'exécution complète de la boucle, il ne reste qu'un seul bulletin dans la liste `urne`. De plus, l'invariant prouvé au point précédent et qui doit être respecté également après l'exécution complète de la boucle permet de conclure que :

- si le dernier bulletin est en faveur de B, alors le nombre de B est impair et a toujours été un nombre impair. La règle (\*) est donc respectée si le nombre initial de B est impair ;
- sinon, une analyse similaire montre que A est élu, le nombre de B est nul et le nombre initial de B devait être pair. La règle (\*) est également respectée si le nombre initial de B est pair.

### Erreurs fréquentes :

- Preuve d'arrêt.*
  - Ne pas justifier pourquoi il y a un élément en moins à chaque itération.
  - Ne pas justifier dans quel cas la fonction s'arrête (en précisant qu'il n'y a qu'une boucle et quelle est sa condition).
- Preuve de l'invariant.*
  - Ne pas prouver la validité de l'invariant *avant* d'entrer dans la boucle.
  - Utiliser un mauvais "cas de base" (preuve *avant*) en supposant que la liste a 1 ou 2 éléments : ici on devait justement supposer que la liste était encore inchangée et donc complète.
  - Mauvaise compréhension de ce que représentent  $x$  et  $y$ .
  - Preuve par l'exemple au lieu d'une preuve en toute généralité (= zéro pour la question).
  - Oubli d'un des 3 cas possibles.
  - Profusion inutile de différents cas en fonction de la parité de  $x$  (le fait que  $x$  soit pair ou impair ne change rien à la preuve).
- Preuve de validité de la règle (\*).*
  - Ne pas justifier sa réponse en s'appuyant explicitement sur ce qui a été prouvé aux **deux** points précédents (et pas uniquement l'invariant). Ceux-ci sont nécessaires pour argumenter ce qui se passe après l'exécution complète de la boucle.
  - La règle (\*) parle de la parité du nombre initial de B, et jamais du nombre de A. On ne peut donc pas dire "si le nombre de A est pair, alors A l'emporte" : ce n'est pas toujours vrai (en réalité, le résultat de l'élection est totalement indépendant des votes en faveur de A).